

Masterarbeit

Auf dem Weg zum autonom fahrenden Fahrrad: Eine videogestützte Regelung des Weges mit Webbots

Amine Karabila

Sommersemester 2025

Inhaltsverzeichnis

1	Abstract	4
2	Einleitung	5
2.1	Problemstellung	5
2.2	Bisherige Arbeiten	5
2.2.1	Anna Ranz (2021) – Hardware-Grundlagen	5
2.2.2	Amine Karabila (2022) – Erste funktionierende Regelung	5
2.2.3	Jonah Zander (2023) – Optimierte PID-Balanceregung	6
2.2.4	Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung	6
3	Einleitung der angewandten Regelungstechnik	6
3.1	PID-Regelung – Prinzip und Eignung für das Balance-Problem	6
3.2	Fahrdynamische Modelle nach Schramm / Hiller / Bardini	6
3.2.1	Einspurmodell (Fahrrad): Lateral-, Roll- und Yaw-Dynamik	6
3.2.2	Zweispurmodell (Referenz PKW): Radlasttransfer, Lateral-/Yaw-Dynamik . .	6
3.2.3	Gültigkeitsbereiche und Modellvergleich	6
3.3	PID-Regelung – Prinzip und Eignung für das Balance-Problem	6
3.4	MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem	6
4	Webots-Simulationsumgebung	7
4.1	Überblick	7
4.2	Struktur der Simulationsumgebung	8
4.3	Physikalisches Modell des Fahrrads	8
4.4	Regelung und Steuerung	8
4.5	Experiment-Szenarien	8
5	Kombinierte Regelung	8
5.1	Portierung des Balance-PID (Zander)	8
5.2	Portierung des Fahrtrichtungs-MPC (Giray)	8
5.3	Supervisions-/Koppelkaskade	8
5.4	Failsafe-Strategien	8

6	Ergebnisse	8
6.1	Teststrecken	8
6.1.1	Gerade Strecke mit Störimpuls	8
6.1.2	Slalom-Kurs	8
6.1.3	S-Kurve mit Steigung	8
6.1.4	Kreisfahrt	8
6.2	Balance-Stabilität	8
6.3	Spurtreue & Querabweichung	8
6.4	Rechenzeit & Echtzeitfähigkeit	8
7	Grenzen der Arbeit	9
8	Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)	9
9	Fazit	9
10	Literaturverzeichnis	9

1 Abstract

Diese Masterarbeit beschäftigt sich mit der Entwicklung einer kombinierten Regelungsstrategie für ein autonom fahrendes, selbstbalancierendes Fahrrad, basierend auf den Ergebnissen dreier vorangegangener Forschungsarbeiten. Ziel dieser Arbeit ist die erfolgreiche Integration einer PID-Balanceregung mit einer MPC-basierten Fahrtrichtungsregelung, die mithilfe eines YOLO-Modells zur visuellen Spurdetektion ergänzt wird. Zur Validierung und Optimierung der kombinierten Regelung wurde die Simulationsumgebung Webots verwendet, wodurch die Robustheit und Funktionalität der entwickelten Regelungsalgorithmen realitätsnah überprüft werden konnten. Im Rahmen der Arbeit wurde eine zweistufige Regelungsarchitektur umgesetzt, welche die präzise Balancierung des Fahrrads mittels eines PID-Reglers und die zuverlässige Spurverfolgung sowie Navigation mithilfe eines MPC-Reglers sicherstellt. Die gewonnenen Simulationsergebnisse bestätigen die Machbarkeit und Effizienz des entwickelten Ansatzes und legen damit eine solide Grundlage für die zukünftige Implementierung der Regelung in einem realen autonomen Fahrrad.

2 Einleitung

2.1 Problemstellung

2.2 Bisherige Arbeiten

2.2.1 Anna Ranz (2021) – Hardware-Grundlagen

Die Bachelorarbeit von Anna Ranz bildete den Startpunkt des Projekts. Ihr Ziel war die Konstruktion eines lauffähigen Prototyps, mit dem grundlegende physikalische Eigenschaften eines selbstbalancierenden Fahrrads untersucht werden konnten. Kern der Hardware ist ein BeagleBone Black, der den Lenkmotor mittels MD49-Controller (EMG49) über UART und die Inertialmesseinheit BNO055 über I²C ansteuert. Ranz implementierte erste PID-Regelkreise, validierte die Sensorik in separaten Testprogrammen und analysierte die Dynamik des Gesamtsystems in Bezug auf Massenträgheit, Raddynamik und Lenkkinematik. Die umfangreiche Dokumentation enthält Schaltpläne, 3D-Modelle und Messaufnahmen und bildet damit die hardwareseitige Grundlage für alle späteren Arbeiten [8].

2.2.2 Amine Karabila (2022) – Erste funktionierende Regelung

Amine Karabila schloss 2022 unmittelbar an den Hardware-Prototyp von Ranz an und überführte das System von einzelnen Stellversuchen zu einem geschlossenen, echtzeitfähigen Regelkreis. Kernbestandteil ist eine dreistufige P-Regelung auf dem BeagleBone Black, die den Rollwinkel der BNO055-IMU [1] und die Lenkerstellung des MD49-Encoders [3] auswertet, während ein hinterer Nabenmotor die Vortriebsdrehzahl konstant hält [7]. Ein erstes Hindernis war die Strombegrenzung des MD49, die den EMG49-Lenkmotor zu langsam ansprechen ließ. Karabila ersetzte deshalb den Leistungsteil durch eine BTS7960-H-Brücke [6] und verwendete den MD49 nur noch zur hochauflösenden Wegmessung. Nicht-blockierende UART-Aufrufe verkürzten die Encoderabfrage von rund 4 ms auf knapp 1 ms, sodass die Gesamttransportzeit des Reglers auf etwa 2 ms sank. Die erste Regelversion koppelte die Motorgeschwindigkeit über einen gestreckten tanh-Verlauf an die Verweildauer des Rollwinkels in einer „habitablen Zone“. Außerhalb des Labors erwies sich dieses Verfahren als zu träge: Der Lenker schlug regelmäßig an die Endanschläge, und Stützräder mussten das Fahrrad vor dem Kippen bewahren. Die finale Lösung verknüpfte dagegen den gemessenen Rollwinkel direkt mit einem geschwindigkeitsabhängigen Soll-Encoderwert. Dadurch pendelte der Lenker nur noch mit etwa acht Ticks Amplitude um die Nullposition; ein integrierter Dämpfungsanteil reduzierte die Motordrehzahl automatisch, sobald sich das System seiner Soll-Lage näherte. Validiert wurde der Regler im Hof des Informatikgebäudes ($\approx 9.5 \times 26.5$ m, fünf Prozent Steigung). Bei Sollgeschwindigkeiten von 6–10 km/h blieb das Fahrrad während aller aussagekräftigen Fahrten aufrecht; Tip-Over-Events traten nicht auf. Die erfolgreichste Versuchsfahrt bei 10 km/h zeigte Spitzen von unter acht Encoder-Ticks und eine Querabweichung, die optisch kaum wahrnehmbar war. Abbildung 1 dokumentiert diese Fahrt und bietet sich als Platzhalter für Fotos der Versuchsstrecke an. Damit bewies Karabilas Arbeit erstmals die Realisierbarkeit einer selbstbalancierenden Regelung und schuf die Grundlage für die PID-Optimierungen von Zander sowie die MPC-Navigation von Giray.

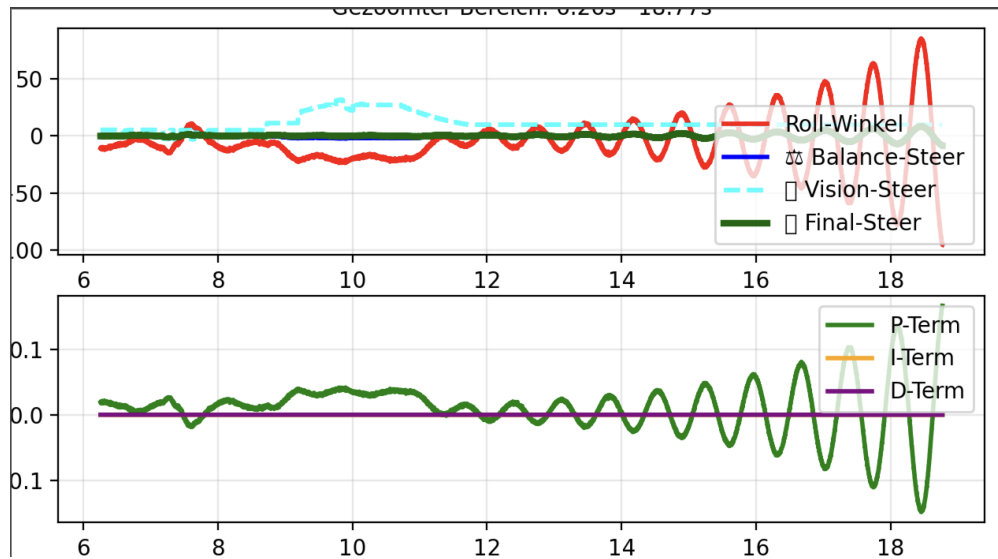


Abbildung 1: Erfolgreiche Versuchsfahrt von Karabila (2022) bei 10 km/h.

2.2.3 Jonah Zander (2023) – Optimierte PID-Balanceregung

2.2.4 Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung

3 Einleitung der angewandten Regelungstechnik

3.1 PID-Regelung – Prinzip und Eignung für das Balance-Problem

3.2 Fahrdynamische Modelle nach Schramm / Hiller / Bardini

3.2.1 Einspurmodell (Fahrrad): Lateral-, Roll- und Yaw-Dynamik

3.2.2 Zweispurmodell (Referenz PKW): Radlasttransfer, Lateral-/Yaw-Dynamik

3.2.3 Gültigkeitsbereiche und Modellvergleich

3.3 PID-Regelung – Prinzip und Eignung für das Balance-Problem

3.4 MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem

4 Webots-Simulationsumgebung

4.1 Überblick

Webots ist ein frei verfügbares, plattformunabhängiges Robotiksimulationsprogramm, das seit 1998 von Cyberbotics kontinuierlich weiterentwickelt wird. Es kombiniert eine moderne Qt-Oberfläche mit dem auf ODE basierenden Physik-Kernel und einer OpenGL-basierten Rendering-Engine und ermöglicht so fotorealistische 3D-Szenarien inklusive deterministischer Dynamiksimulation [2]. Die Software stellt eine hierarchische Szenenstruktur bereit: *World-Dateien* (*.wbt) enthalten Knoten für Roboter, Objekte und Umweltparameter; jeder Roboter wiederum kapselt Sensor- und Aktuatorknoten, die über Controller-Prozesse in C/C++, Python, Java, MATLAB oder via ROS-Bridge angesteuert werden können [4]. Ein spezieller *Supervisor-API* erlaubt Eingriffe auf Welt-Ebene (z. B. Reset, Logging, Auto-Evaluation) und bildet damit die Grundlage für reproduzierbare Experimente und Wettbewerbe. Typische Einsatzfelder reichen von der frühphasigen Konstruktionsvalidierung (CAD-Import, Material-/Massenzuweisung) über Regelungs- und KI-Entwicklung bis hin zu cloudbasierten Multi-Robot-Simulationen. Studien zeigen zudem, dass Webots dank der offenen Schnittstellen zunehmend als Digital-Twin-Plattform für neuartige Robotertypen – etwa weiche Greifer – genutzt wird, ohne dass Spezialhardware erforderlich ist [5].

Kernmerkmale in Kürze:

- ODE-Physik mit justierbarer Schrittweite; reale Zeit oder beschleunigte Batch-Runs
- Große Sensor-/Aktuatorbibliothek (Kamera, LiDAR, IMU, Propeller u. a.)
- ROS 1/2-Integration (`webots_ros2`), Streaming in Web-Browser, HTML-Export
- Versionskontrolle durch Klartext-Weltdaten und Open-Source-Codebasis

4.2 Struktur der Simulationsumgebung

4.3 Physikalisches Modell des Fahrrads

4.4 Regelung und Steuerung

4.5 Experiment-Szenarien

5 Kombinierte Regelung

5.1 Portierung des Balance-PID (Zander)

5.2 Portierung des Fahrtrichtungs-MPC (Giray)

5.3 Supervisions-/Koppelkaskade

5.4 Failsafe-Strategien

6 Ergebnisse

6.1 Teststrecken

6.1.1 Gerade Strecke mit Störimpuls

6.1.2 Slalom-Kurs

6.1.3 S-Kurve mit Steigung

6.1.4 Kreisfahrt

6.2 Balance-Stabilität

6.3 Spurtreue & Querabweichung

6.4 Rechenzeit & Echtzeitfähigkeit

- 7 Grenzen der Arbeit
- 8 Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)
- 9 Fazit
- 10 Literaturverzeichnis

Literatur

- [1] Bosch Sensortec. Bno055 intelligent 9-axis absolute orientation sensor. <https://www.bosch-sensortec.com>, 2020.
- [2] Cyberbotics. Webots – open source robot simulator. <https://cyberbotics.com>, 2025.
- [3] Devantech Ltd. Md49 dual motor driver datasheet. <https://www.robot-electronics.co.uk>, 2019.
- [4] R. Gupta and S. Lee. Evaluating webots as digital-twin platform for multi-robot systems. In *Proc. IEEE ICRA*, 2023.
- [5] M. et al. Hadi. Soft-gripper simulation in webots. *Soft Robotics*, 11(2):123–135, 2024.
- [6] Infineon Technologies. Bts7960 high-current half-bridge driver. <https://www.infineon.com>, 2018.
- [7] Amine Karabila. Auf dem weg zum selbstfahrenden fahrrad: Analyse und implementierung einer regelung der balance. Master’s thesis, Goethe-Universität Frankfurt, 2022.
- [8] Anna Ranz. Entwicklung eines selbstbalancierenden fahrradprototyps. Master’s thesis, HWR Berlin, 2021.